

June 2026 Plan — RTL build + bit-exact co-simulation

Aligned with the research plan (§9.3): implement `item_memory`, `bundle_unit`, `pruning_mask`, `popcount_am` RTL + co-sim passing; D parameterisation verified.

Goal (one sentence)

Implement the four new RTL modules and have them pass **bit-exact co-simulation** against `hdc_ref.py`, with **D** and **bundle counter width (CNT_W)** parameterisation verified — no board, no power, no experiments yet.

Starting point (already done in May)

- `xor_permute_top.sv` + `permute_stage.sv` + self-checking testbench.
- `hdc_ref.py` golden reference (smoke tests pass).
- 1000 bind+permute co-sim vectors in `python_ref/vectors/`.
- Item-memory and example pruning-mask `.mem` files in `python_ref/mem_files/`.
- Frozen software baseline: 90.30 % ± 0.13 spatial at D=1024 (protocol P-may2026).

Week-by-week

Week 1 — Co-sim harness + `item_memory`

#	Task	Output
J1	Automated co-sim harness: TB reads <code>vectors/*.hex</code> + <code>mem_files/</code> , runs RTL, compares to golden, prints PASS/FAIL; wrap as one <code>.do</code> script	<code>tb/cosim*.sv</code> , <code>sim/run_cosim.do</code>
J2	<code>item_memory.sv</code> — channel iM + continuous level CiM via <code>\$readmemh</code> , parameterised <code>D</code> (replaces <code>simple_bind_rom</code> placeholder)	<code>rtl/item_memory.sv</code>
J3	TB: <code>item_memory</code> lookups vs <code>hdc_ref</code> exported memories, bit-exact	<code>tb/tb_item_memory.sv</code>

Week 2 — `bundle_unit` + `popcount_am`

#	Task	Output
J4	<code>bundle_unit.sv</code> — thresholded majority, tie→0, parametric <code>CNT_W</code>	<code>rtl/bundle_unit.sv</code>

#	Task	Output
J5	<code>popcount_am.sv</code> — masked Hamming distance to K class prototypes + argmin; parametric <code>D</code> , <code>K</code>	<code>rtl/popcount_am.sv</code>
J6	Extend <code>generate_vectors.py</code> to emit bundle + AM cases; TBs vs golden	new vectors + 2 TBs

Week 3 — `pruning_mask` + integration

#	Task	Output
J7	<code>pruning_mask.sv</code> — per-bit AND before popcount (mask from <code>mem_files/</code>)	<code>rtl/pruning_mask.sv</code>
J8	Integrate bind → permute → bundle → (mask) → popcount into a classify-pipeline top	<code>rtl/hdc_classify_top.sv</code>
J9	End-to-end co-sim: encode EMG windows in Python, compare RTL decisions bit-exactly at D=1024	end-to-end TB + report

Week 4 — D parameterisation + regression

#	Task	Output
J10	Verify <code>D ∈ {256, 512, 1024, 2048}</code> and <code>CNT_W</code> variants match golden	sweep log
J11	One-command regression (compile + all TBs + end-to-end → PASS/FAIL)	<code>sim/run_regression.do</code>
J12	Tag June milestone; write June results PDF note	<code>python_ref/notes/june_cosim.pdf</code>

Explicitly NOT in June

- **July:** `hdc_stream_wrapper.sv` (AXI-Stream + DMA) and board bring-up.
- **August:** Hook A Pareto + Twist 1 (informed vs random) + Twist 2 (cross-subject) + power.
- **September:** paper draft → DATE submission.

"June done" checklist

- [] `item_memory`, `bundle_unit`, `pruning_mask`, `popcount_am` implemented
- [] each passes bit-exact co-sim vs `hdc_ref.py`
- [] full pipeline matches Python decisions at D=1024
- [] D parameterisation verified across ≥3 values
- [] one-command regression passes